



Vidyavardhini's College of Engineering and Technology

Department of Artificial Intelligence & Data Science

AY: 2026-27

Class:	TE-AI&DS	Semester:	V
Course Code:	2015111	Course Name:	Statistics for Machine Learning and Data Science Lab

Name of Student:	ASHLESHA .S. BHOIR
Roll No. :	03
Experiment No.:	7
Title of the Experiment:	Build and Evaluate Classification Models using Supervised Learning Algorithms
Date of Performance:	2-10-2026
Date of Submission:	9-10-2026

Evaluation

Performance Indicator	Max. Marks	Marks Obtained
Performance	5	
Understanding	5	
Journal work and timely submission	10	
Total	20	

Performance Indicator	Exceed Expectations (EE)	Meet Expectations (ME)	Below Expectations (BE)
Performance	4-5	2-3	1
Understanding	4-5	2-3	1
Journal work and timely submission	8-10	5-8	1-4

Checked by

Name of Faculty : Dr. Tatwadarshi Nagarhalli

Signature :

Date :



Vidyavardhini's College of Engineering and Technology

Department of Artificial Intelligence & Data Science

Aim: To build supervised classification models for predicting diabetes outcomes using the Pima Indians Diabetes Dataset and evaluate their performance using appropriate classification metrics.

Objectives

After completing this experiment, the student will be able to:

1. Build supervised classification models to predict the diabetes outcome of patients.
2. Evaluate and interpret classification model performance using appropriate evaluation metrics.

Tools Required

- Python 3.x
- Google Colab / Jupyter Notebook
- Pandas
- NumPy
- Matplotlib
- Seaborn
- Scikit-learn

Dataset Details

Dataset: Pima Indians Diabetes Dataset

The dataset contains medical information for **768 female patients**.

The target variable is:

- Outcome = 0: Non-diabetic
- Outcome = 1: Diabetic



Vidyavardhini's College of Engineering and Technology

Department of Artificial Intelligence & Data Science

Procedure

1. Load the Pima Indians Diabetes Dataset.
2. Separate the input features and target variable.
3. Identify and handle invalid or missing values.
4. Divide the dataset into training and testing sets using a stratified split.
5. Apply suitable preprocessing, such as feature scaling where required.
6. Build one or more classification models, such as:
 - Logistic Regression
 - k-Nearest Neighbors
 - Decision Tree
7. Train the selected model(s) using the training data.
8. Generate predictions for the test data.
9. Construct the confusion matrix.
10. Calculate classification performance metrics.
11. Compare the obtained results and interpret the model performance.

Description of the Experiment

Classification is a supervised learning technique used to assign observations to predefined categories.

In this experiment, the objective is to classify a patient as: Patient Health Data, Classification Model gives Non-Diabetic and Diabetic.



Vidyavardhini's College of Engineering and Technology

Department of Artificial Intelligence & Data Science

The experiment follows: Dataset, Feature and Target Selection, Data Preprocessing, Train-Test Split, Classification Model, Prediction, and Performance Evaluation

Detailed Description of Statistical Methods

Logistic Regression

Logistic regression predicts the probability of a binary outcome.

The logistic function is:

$$P(Y = 1) = \frac{1}{1+e^{-z}}$$

where:

$$z = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \dots + \beta_p x_p$$

The predicted probability is converted into a class using a decision threshold.

Confusion Matrix

A confusion matrix contains:

Predicted Negative Predicted Positive

Actual Negative True Negative (TN) False Positive (FP)

Actual Positive False Negative (FN) True Positive (TP)

Accuracy

$$Accuracy = \frac{TP+TN}{TP+TN+FP+FN}$$

Precision

$$Precision = \frac{TP}{TP+FP}$$

Precision indicates how many predicted positive cases were actually positive.



Recall

$$Recall = \frac{TP}{TP+FN}$$

Recall indicates how many actual positive cases were correctly identified.

F1-Score

$$F1 = 2 \left(\frac{Precision \times Recall}{Precision + Recall} \right)$$

The F1-score balances precision and recall.

ROC-AUC

The ROC curve represents the relationship between:

- True Positive Rate
- False Positive Rate

The Area Under the Curve (AUC) provides an overall measure of the model's ability to distinguish between the two classes.

Implementation:

```
[1] ✓ 2s  import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import (
    accuracy_score,
    precision_score,
    recall_score,
    f1_score,
    confusion_matrix,
    classification_report,
    roc_auc_score,
    roc_curve
)
```



Vidyavardhini's College of Engineering and Technology

Department of Artificial Intelligence & Data Science

```
[2]
✓ Os ▶ url = "https://raw.githubusercontent.com/jbrownlee/Datasets/master/pima-indians-diabetes.data.csv"

columns = [
    "Pregnancies",
    "Glucose",
    "BloodPressure",
    "SkinThickness",
    "Insulin",
    "BMI",
    "DiabetesPedigreeFunction",
    "Age",
    "Outcome"
]

df = pd.read_csv(url, names=columns)

df.head()
```

	Pregnancies	Glucose	BloodPressure	SkinThickness	Insulin	BMI	DiabetesPedigreeFunction	Age	Outcome
0	6	148	72	35	0	33.6	0.627	50	1
1	1	85	66	29	0	26.6	0.351	31	0
2	8	183	64	0	0	23.3	0.672	32	1
3	1	89	66	23	94	28.1	0.167	21	0
4	0	137	40	35	168	43.1	2.288	33	1

```
[3]
✓ Os ▶ print("Dataset Shape:", df.shape)
print("\nDataset Information:")
df.info()
```

Dataset Shape: (768, 9)

Dataset Information:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 768 entries, 0 to 767
Data columns (total 9 columns):

#	Column	Non-Null Count	Dtype
0	Pregnancies	768 non-null	int64
1	Glucose	768 non-null	int64
2	BloodPressure	768 non-null	int64
3	SkinThickness	768 non-null	int64
4	Insulin	768 non-null	int64
5	BMI	768 non-null	float64
6	DiabetesPedigreeFunction	768 non-null	float64
7	Age	768 non-null	int64
8	Outcome	768 non-null	int64

dtypes: float64(2), int64(7)
memory usage: 54.1 KB



Vidyavardhini's College of Engineering and Technology

Department of Artificial Intelligence & Data Science

[4]

✓ 0s

```
print("Zero values in each column:")  
print((df == 0).sum())
```

... Zero values in each column:

Pregnancies	111
Glucose	5
BloodPressure	35
SkinThickness	227
Insulin	374
BMI	11
DiabetesPedigreeFunction	0
Age	0
Outcome	500

dtype: int64

[5]

✓ 0s

```
invalid_columns = [  
    "Glucose",  
    "BloodPressure",  
    "SkinThickness",  
    "Insulin",  
    "BMI"  
]  
  
df[invalid_columns] = df[invalid_columns].replace(0, np.nan)  
  
print(df.isnull().sum())
```

... Pregnancies 0
Glucose 5
BloodPressure 35
SkinThickness 227
Insulin 374
BMI 11
DiabetesPedigreeFunction 0
Age 0
Outcome 0
dtype: int64

[6]

✓ 0s

```
for column in invalid_columns:  
    df[column] = df[column].fillna(df[column].median())  
  
print("Missing values after preprocessing:")  
print(df.isnull().sum())
```

... Missing values after preprocessing:

Pregnancies	0
Glucose	0
BloodPressure	0
SkinThickness	0
Insulin	0
BMI	0
DiabetesPedigreeFunction	0
Age	0
Outcome	0

dtype: int64



Vidyavardhini's College of Engineering and Technology

Department of Artificial Intelligence & Data Science

[7]
✓ Os

```
X = df.drop("Outcome", axis=1)
y = df["Outcome"]
```

```
print("Features:")
print(X.columns)
```

```
print("\nTarget:")
print(y.name)
```

✓

```
... Features:
Index(['Pregnancies', 'Glucose', 'BloodPressure', 'SkinThickness', 'Insulin',
       'BMI', 'DiabetesPedigreeFunction', 'Age'],
      dtype='object')
```

```
Target:
Outcome
```

[8]
✓ Os

```
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.20,
    random_state=42,
    stratify=y
)
```

```
print("Training data:", X_train.shape)
print("Testing data:", X_test.shape)
```

✓

```
Training data: (614, 8)
Testing data: (154, 8)
```

[9]
✓ Os

```
scaler = StandardScaler()
```

[10]
✓ Os

```
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)
```

[15]
✓ Os

```
model = LogisticRegression(random_state=42)
```



Vidyavardhini's College of Engineering and Technology

Department of Artificial Intelligence & Data Science

[18]
✓ Os

```
model.fit(X_train_scaled, y_train)
```



```
LogisticRegression  
LogisticRegression(random_state=42)
```

[19]
✓ Os

```
y_pred = model.predict(X_test_scaled)  
  
print("Predicted values:")  
print(y_pred)
```



```
... Predicted values:  
[1 0 0 0 0 0 0 1 0 1 0 0 0 0 0 0 1 0 1 0 0 1 0 1 1 0 1 0 0 0 0 0 0 1 1 0 0  
0 1 1 0 0 0 0 0 0 0 0 1 0 1 1 0 0 1 0 1 0 1 0 0 1 0 0 1 0 0 1 0 0 0 0 0  
0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 1 1 0 0 1 1 0 1 1 0 0 0 0 0 1 0 1 0 1 0 1  
1 0 0 0 0 0 0 1 0 1 0 0 1 0 1 1 1 0 0 0 1 0 1 1 0 0 0 0 0 0 0 0 0 0 0 0 0 1  
0 0 0 0 1 0]
```

[20]
✓ Os

```
y_prob = model.predict_proba(X_test_scaled)[: , 1]  
  
print("Predicted probabilities:")  
print(y_prob[:10])
```



```
... Predicted probabilities:  
[0.60971073 0.11835576 0.29346313 0.2508367 0.03320667 0.16852385  
0.4881643 0.92367511 0.08143405 0.8385896 ]
```

[21]
✓ Os

```
cm = confusion_matrix(y_test, y_pred)  
  
print("Confusion Matrix:")  
print(cm)
```



```
Confusion Matrix:  
[[82 18]  
 [27 27]]
```

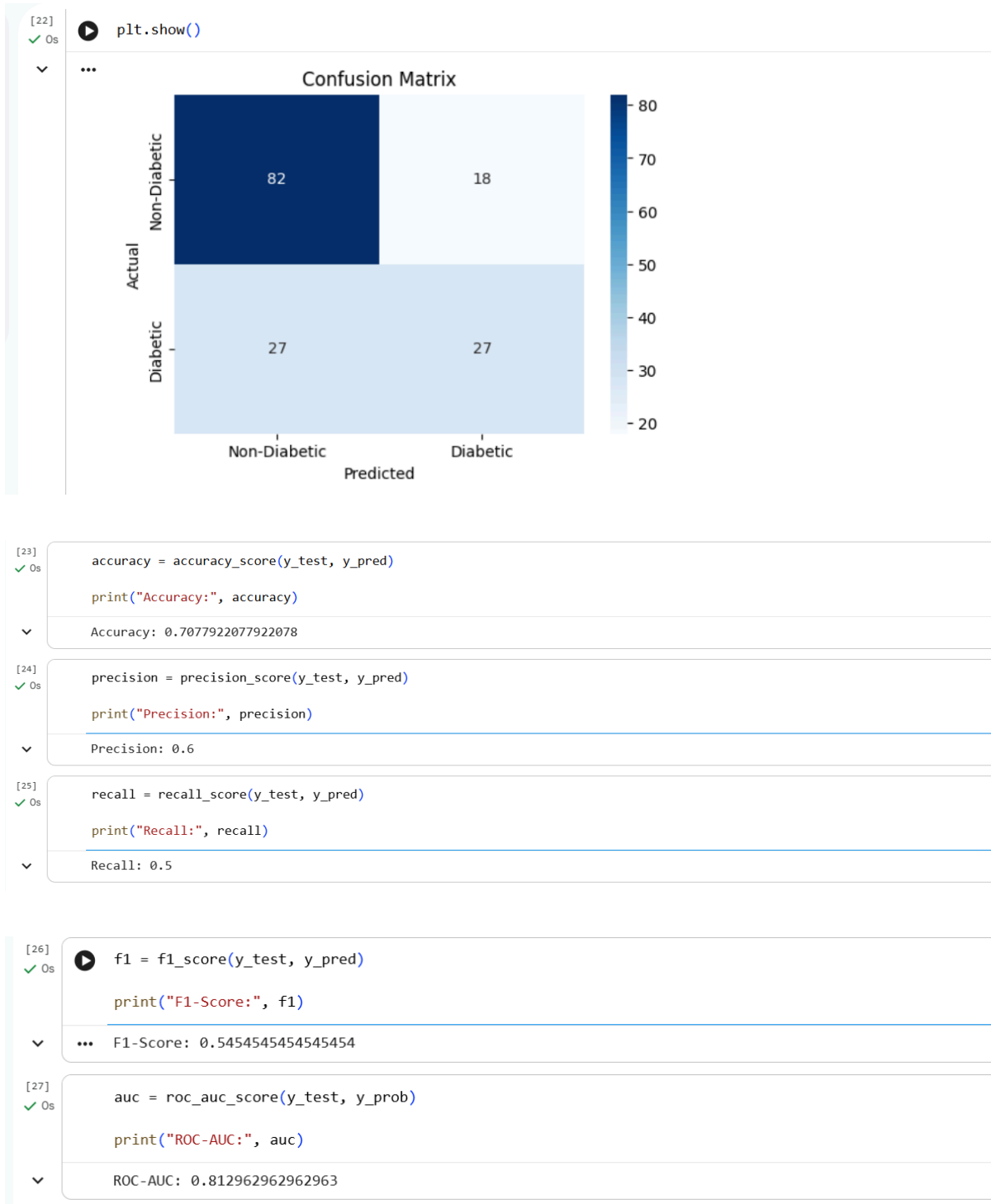
[22]
✓ Os

```
plt.figure(figsize=(6, 4))  
  
sns.heatmap(  
    cm,  
    annot=True,  
    fmt="d",  
    cmap="Blues",  
    xticklabels=["Non-Diabetic", "Diabetic"],  
    yticklabels=["Non-Diabetic", "Diabetic"]  
)  
  
plt.xlabel("Predicted")  
plt.ylabel("Actual")  
plt.title("Confusion Matrix")  
  
plt.show()
```



Vidyavardhini's College of Engineering and Technology

Department of Artificial Intelligence & Data Science





[28]
✓ 0s

```
print("Classification Model Performance")
print("-----")
print("Accuracy :", accuracy)
print("Precision :", precision)
print("Recall :", recall)
print("F1-Score :", f1)
print("ROC-AUC :", auc)
```

✓

```
... Classification Model Performance
-----
Accuracy : 0.7077922077922078
Precision : 0.6
Recall : 0.5
F1-Score : 0.5454545454545454
ROC-AUC : 0.812962962962963
```

[29]
✓ 0s

```
print("Classification Report:")
print(classification_report(y_test, y_pred))
```

✓

```
... Classification Report:
              precision    recall  f1-score   support

     0       0.75      0.82      0.78      100
     1       0.60      0.50      0.55       54

 accuracy          0.68
 macro avg         0.68      0.66      0.67
weighted avg         0.70      0.71      0.70
```

[30]
✓ 0s

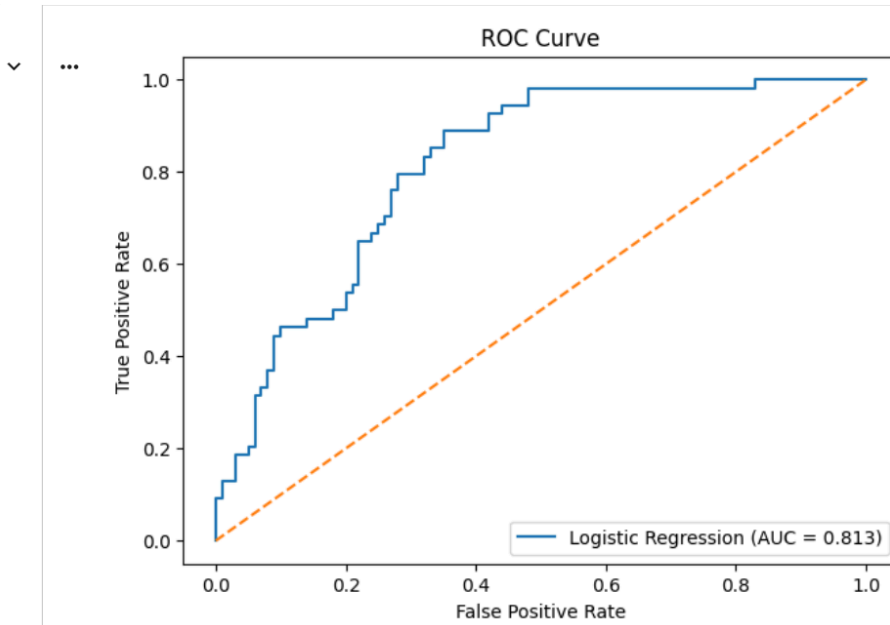
```
fpr, tpr, thresholds = roc_curve(y_test, y_prob)

plt.figure(figsize=(7, 5))

plt.plot(fpr, tpr, label=f"Logistic Regression (AUC = {auc:.3f})")
plt.plot([0, 1], [0, 1], linestyle="--")

plt.xlabel("False Positive Rate")
plt.ylabel("True Positive Rate")
plt.title("ROC Curve")
plt.legend()

plt.show()
```



Conclusion

Supervised classification models were developed to predict diabetes outcomes using the Pima Indians Diabetes Dataset. The models were evaluated using the confusion matrix, accuracy, precision, recall, F1-score, and ROC-AUC. The experiment demonstrated that different evaluation metrics provide different perspectives on classification performance.

Questions to be Answered by Students

1. Which classification model was implemented, and what were its accuracy, precision, recall, and F1-score?

ANS: The Logistic Regression model was implemented to classify patients into diabetic and non-diabetic categories.

Classification Model Performance:

Accuracy : 0.7077922077922078

Precision : 0.6

Recall : 0.5



Vidyavardhini's College of Engineering and Technology

Department of Artificial Intelligence & Data Science

F1-Score : 0.5454545454545454

ROC-AUC : 0.812962962962963

2. How many true positives and false negatives were obtained from the confusion matrix?

ANS: The confusion matrix represents the actual and predicted classes. From the confusion matrix:

True Positive (TP): 27

False Negative (FN): 27

A True Positive represents a diabetic patient correctly classified as diabetic, while a False Negative represents a diabetic patient incorrectly classified as non-diabetic.

3. Which metric is more important for diabetes prediction: precision or recall? Justify your answer.

ANS: Recall is more important for diabetes prediction because it measures the proportion of actual diabetic patients who are correctly identified by the model.

A false negative occurs when a patient who is actually diabetic is classified as non-diabetic. A higher recall reduces these missed diabetic cases and therefore helps identify more patients who may require further medical evaluation.

Precision is also important because it measures how many patients predicted as diabetic are actually diabetic, but recall is particularly relevant when the objective is to minimize missed positive cases.

4. What was the ROC-AUC value, and what does it indicate about the model's performance?

ANS: The **ROC-AUC value** obtained from the Logistic Regression model was:

ROC-AUC = 0.812962962962963



Vidyavardhini's College of Engineering and Technology

Department of Artificial Intelligence & Data Science

ROC-AUC measures the model's ability to distinguish between diabetic and non-diabetic patients over different classification thresholds.

- An **AUC close to 1** indicates stronger discrimination.
- An **AUC around 0.5** indicates performance close to random classification.
- A value between these indicates the degree to which the model can distinguish the two classes.

Therefore, the obtained ROC-AUC value indicates how effectively the developed Logistic Regression model separates diabetic patients from non-diabetic patients.

5. State **two limitations** of the developed classification model.

ANS: Linear assumption: Logistic Regression assumes a linear relationship between the input features and the log-odds of the diabetes outcome. Complex or nonlinear relationships may not be fully captured by the model.

Data quality and generalization: The model is trained on the Pima Indians Diabetes Dataset, which contains a specific population and some invalid/missing medical values. Therefore, its performance may not generalize directly to other populations or datasets.